

Routing Strategies for Ring and General Topologies in OMNeT++

Bosque, Lissandro; Galassi, Franco; Pairetti, Joaquín

Grupo 31, Facultad de Matemática, Astronomía, Física y Computación (FaMAF)
Universidad Nacional de Córdoba, Argentina

Abstract

Write a concise abstract in English (150–250 words). Summarize the simulated network scenarios, the routing algorithm you designed, the main performance metrics (delay, hop count, buffer/link utilization, stability), and the key findings when comparing your solution against the kickstarter baseline. Mention whether routing loops were observed and, if applicable, how your algorithm generalizes to arbitrary topologies.

Index Terms— Network simulation, OMNeT++, routing, ring topology, performance evaluation, queueing, stability.

1 Introducción

Este artículo reporta el análisis y diseño de estrategias de enrutamiento implementadas sobre el modelo de red del Laboratorio 4. Fue desarrollado por el grupo de forma tal que se priorizó la discusión total de su contenido, para luego dividir los temas y que cada integrante redacte un texto base para luego compartirlo a los demás. Por último, se hizo una revisión y corrección de errores.

A través de un *kickstarter*, provisto por la cátedra de Redes 2026, nos fue dada una implementación base de un modelo de red de 8 nodos conectados en forma de anillo con el objetivo de que la misma fuera analizada en OMNeT++ para lograr una mejor comprensión del tráfico de red en el modelo base con unas métricas iniciales y así plantear una forma más óptima de enviar los paquetes implementando cambios en el modelo para lograr el enrutamiento deseado. En el *kickstarter*, con el enrutamiento inicial todos los paquetes son enviados en sentido horario a partir de `toLnk[0]` hasta llegar a su destino, ignorando si se tarda más que enviarlo en sentido antihorario. Por ejemplo, si se tuviere un paquete en el nodo cero, y se quisiera enviarlo hasta el 5, con el modelo inicial se recorrerían 5 nodos. En cambio, si se hiciera con el modelo modificado, se compararía su distancia con el sentido antihorario, el cuál resultaría más eficiente, ya que recorrería 3 nodos.

Resumiendo, el modelo modificado busca cual es la distancia de un nodo a otro en sentido horario y antihorario, decide cual es menor, por lo tanto más eficiente, y recorre los nodos en ese sentido.

2 Escenario y modelo de simulación

2.1 Topología y configuración de tráfico

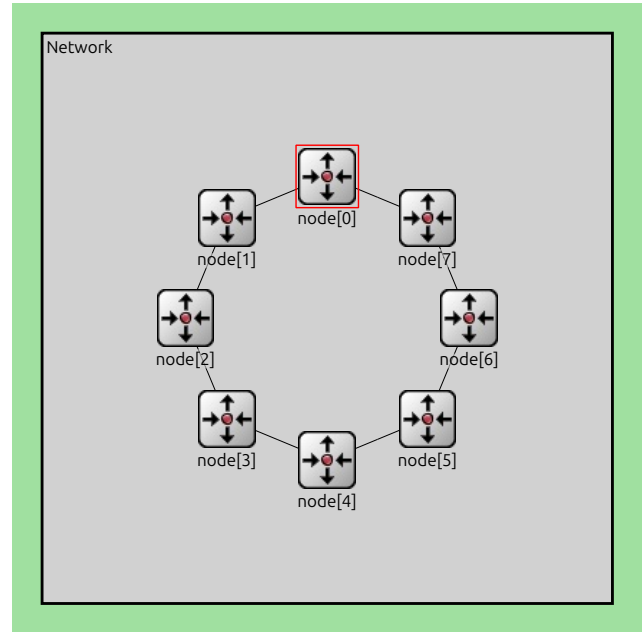


Figure 1: Topología en anillo de 8 nodos graficada en OMNeT++.

El modelo consiste en una red en forma de anillo de 8 nodos implementada en OMNeT++. Cada nodo se conecta con sus dos vecinos mediante enlaces *full-duplex* (bidireccionales) de 1 Mbps de capacidad y 100 μ s de delay, formando un anillo cerrado. Los nodos se indexan del 0 al 7; el nodo `node[1]` tiene como vecinos a `node[0]` (interfaz `lnk[0]`) y a `node[2]` (interfaz `lnk[1]`), y así sucesivamente.

Internamente, cada nodo está compuesto por tres capas:

- **Capa de aplicación (app):** genera paquetes con un tamaño configurable (`packetByteSize`) a intervalos aleatorios distribuidos exponencialmente (`interArrivalTime`), y registra la demora extremo a extremo de los paquetes recibidos.
- **Capa de red (net):** recibe paquetes desde `app` o desde las capas de enlace, y decide por qué interfaz reenviarlos según el algoritmo de enrutamiento implementado.
- **Capas de enlace (lnk[0] y lnk[1]):** modelan un buffer de salida y el canal físico hacia cada vecino.

Como muestra la Fig. 1, el tráfico puede circular en sentido horario (por `lnk[0]`) o antihorario (por `lnk[1]`). El kickstarter utiliza siempre `lnk[0]`, independientemente del destino.

Se definen dos escenarios de tráfico, resumidos en la Tabla 1:

- **Caso 1:** `node[0]` y `node[2]` transmiten paquetes hacia `node[5]`.
- **Caso 2:** los 7 nodos restantes (0–4, 6–7) transmiten simultáneamente hacia `node[5]` con los mismos parámetros de tráfico.

Tabla 1: Configuración experimental por escenario.

Fuentes activas	<code>n[0], n[2]</code>	<code>7 → n[5]</code>	<code>1 y 2</code>
Destino	<code>n[5]</code>	<code>n[5]</code>	<code>n[5]</code>
<code>packetByteSize</code> (B)	125000	125000	125000
<code>interArrivalTime</code>	<code>exp(1)</code>	<code>exp(1)</code>	<code>exp(1)</code>
T. simulación (s)	200	200	200
Semilla	0	0	0

2.2 Métricas registradas

Se instrumentaron las siguientes métricas en el modelo:

- **Demora extremo a extremo:** tiempo transcurrido desde que un paquete es generado en la capa de aplicación del nodo origen hasta que es entregado en el nodo destino. Registrada como vector y escalar en `app`.
- **Cantidad de saltos:** número de nodos intermedios que atraviesa cada paquete. Se incrementa en cada paso por la capa `net` y se registra al llegar al destino.

- **Ocupación del buffer:** cantidad de paquetes en espera en cada `lnk`, registrada como vector temporal. Permite identificar saturación en enlaces específicos.
- **Utilización del enlace:** fracción del tiempo en que el canal estuvo ocupado transmitiendo. Calculada en cada `lnk` como escalar al final de la simulación.

Tabla 2: Métricas instrumentadas en el simulador.

Métrica	Módulo	Unidad
Demora E2E	<code>app</code>	s
Ocupación buffer	<code>lnk[0]</code>	pkts
Saltos/paquete	<code>net</code>	adim.

3 Tarea de análisis: evaluación del kickstarter

Guía para completar: Esta sección responde la **Tarea de Análisis** del enunciado. Presente los Casos 1 y 2 por separado. Cada caso debe incluir: (i) configuración experimental, (ii) tabla o gráfico con resultados, (iii) interpretación basada en datos y (iv) respuesta explícita a las preguntas del enunciado. **No** afirme mejoras o saturación sin mostrar números, curvas o barras comparativas.

3.1 Caso 1: tráfico configurado (`node[0]` y `node[2]` → `node[5]`)

Guía para completar: Ejecute el modelo con la configuración por defecto del kickstarter. Reporte demoras, saltos, uso de buffers y enlaces. Responda: ¿qué métricas obtienen? ¿cómo se usan los recursos? ¿se puede mejorar y por qué?

[Análisis narrativo apoyado en Tabla 3 y Fig. 2. Describa si un sentido del anillo concentra tráfico, si hay nodos intermedios más cargados y si las colas crecen de forma sostenida.]

Guía para completar: Ejemplo de afirmación bien respaldada: “La demora media fue 0.82 s (Tabla 3), un 35% mayor en el flujo desde `node[0]` por la mayor distancia en saltos (Fig. 2).” Evite frases vagas como “la red anduvo bien” sin citar datos.

Table 3: Resultados del Caso 1 (baseline kickstarter). Reemplace los valores.

Métrica	Prom.	Máx.	Desv.
Demora (s)	0.00	0.00	0.00
Salto	0.0	0	0.0
Ocupación buffer	0.0	0	0.0
Util. enlace (%)	0.0	0.0	0.0

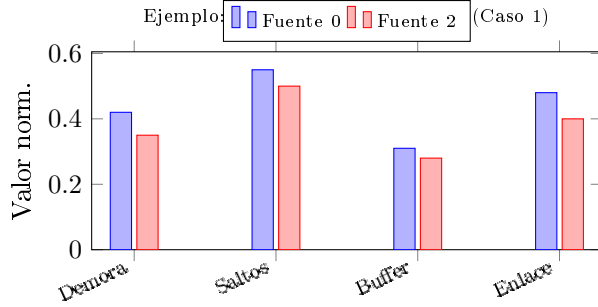


Figure 2: Ejemplo de gráfico de barras. Exporte datos desde OMNeT++ (Scalar Export / Python / R) y genere gráficos similares con valores reales.

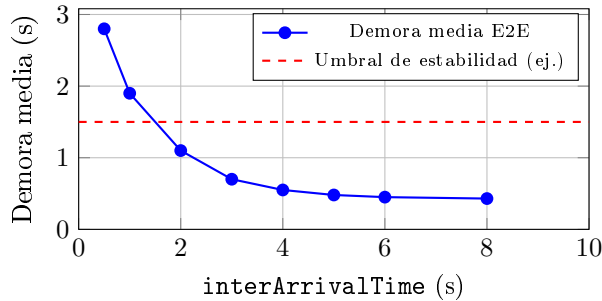


Figure 3: Ejemplo: demora media vs. interArrivalTime. Marque el régimen estable y el de saturación.

3.2 Caso 2: tráfico sostenido hacia node[5]

Guía para completar: Configure todos los nodos (0,1,2,3,4,6,7) enviando a node[5] con el mismo packetByteSize e interArrivalTime. Varíe interArrivalTime sistemáticamente y muestre en un gráfico cuándo la red deja de ser estable (colas crecientes, demoras divergentes, pérdidas). Indique el valor umbral estimado y justifíquelo con evidencia.

[Respuesta explícita: a partir de interArrivalTime = _____ s la red permanece estable porque _____.]

Table 4: Exploración del Caso 2: métricas vs. interArrivalTime.

interArriv.	Demora	Cola	Pérd.	Estab.
0.5	2.80	48	12	No
1.0	1.90	32	3	No
2.0	1.10	15	0	Sí
3.0	0.70	8	0	Sí
5.0	0.48	4	0	Sí
[...]	[...]	[...]	[...]	[...]

Guía para completar: Complete la tabla con sus corridas reales. La columna “¿Estable?” debe basarse en un criterio explícito (p. ej. cola acotada, demora estacionaria, cero pérdidas). Referencie esta tabla al responder el umbral del Caso 2.

4 Diseño del algoritmo de enrutamiento propuesto

Guía para completar: Corresponde a la Tarea de Diseño. Explique qué información usa cada nodo, cómo la obtiene (p. ej. paquetes hello/echo, tablas locales) y cómo decide la interfaz de salida. Describa decisiones de diseño, limitaciones y por qué su enfoque supera al baseline. Puede usar pseudocódigo, diagrama de flujo o tabla de reglas. Recuerde: no use cTopology ni iteradores de cGate para descubrir vecinos.

4.1 Información disponible y estructuras de datos

[Describa tablas de rutas, métricas locales, mensajes de control, etc.]

4.2 Reglas de decisión

Guía para completar: Presente el algoritmo en tres niveles: (1) descripción en prosa, (2) tabla de reglas o pseudocódigo, (3) ejemplo numérico con un paquete concreto que atraviesa varios nodos. Si intercambian mensajes de control, muestre el formato del paquete hello/echo en una tabla.

[Detalle el algoritmo. Ejemplo de pseudocódigo:]

```

al recibir paquete p en interfaz i:
  si dest(p) == id_local:
    entregar a capa app
  sino:
    j <- elegir_interfaz(p, tablas_locales)
    reenviar p por interfaz j

```

Table 5: Ejemplo de tabla de rutas locales en un nodo (complete con su diseño).

Destino	Interfaz	Saltos
node[5]	lnk[1]	2
node[0]	lnk[0]	3
node[2]	lnk[0]	1
[...]	[...]	[...]

5 Evaluación comparativa

Guía para completar: Compare su algoritmo contra el kickstarter en los Casos 1 y 2 (y en otros escenarios si lo desea). Use tablas side-by-side, gráficos de barras agrupados o boxplots de demora. Cuantifique la mejora (% , factor, absoluto). Responda: ¿cuánto mejoran las métricas? ¿por qué? ¿hubo bucles de enrutamiento? ¿Qué pasaría si la cantidad de nodos es mucho mayor y como funcionaría con respecto al algoritmo propuesto por la cátedra? ¿Qué sucedería si se cae un nodo serían tolerantes a esa falla los algoritmos? Si ajustó parámetros según la nota al pie 4 del enunciado, indíquelo claramente en la metodología.

5.1 Metodología de comparación

[Describa bajo qué condiciones ejecutó cada algoritmo, si usó los mismos escenarios, semillas y parámetros, y cómo calculó promedios o intervalos. Ejemplo: “Para cada *interArrivalTime* del Caso 2 se ejecutaron 5 repeticiones con semillas 1–5; reportamos la media y el máximo de demora extremo a extremo.”]

5.2 Resultados cuantitativos

5.3 Discusión de resultados

Guía para completar: Interprete las tablas y figuras anteriores. Relacione mejoras con decisiones concretas de su diseño (p. ej. balanceo de carga, ruta más corta). Mencione mejoras posibles no implementadas por limitaciones de tiempo o alcance.

[Discusión basada en evidencia. ¿Se observaron bucles? ¿En qué condiciones? ¿Cómo los detectó o evitó?]

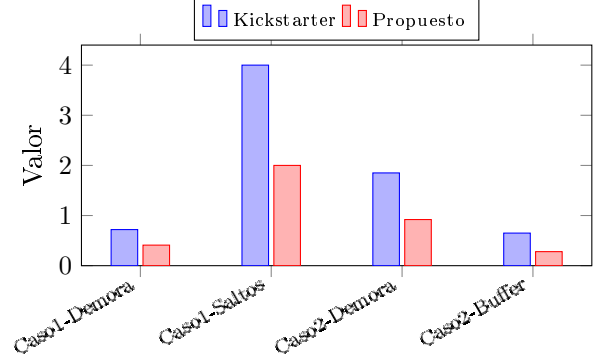


Figure 4: Ejemplo de barras agrupadas para comparar algoritmos. Todo gráfico debe tener ejes etiquetados, unidades y leyenda.

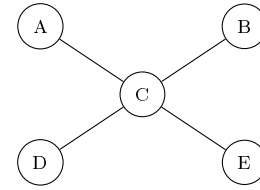


Figure 5: Ejemplo esquemático de topología estrella (*networkStar*). Reemplace por su figura y describa cómo se adapta el algoritmo a N interfaces.

6 Tarea estrella (opcional)

Guía para completar: Si realizaron la tarea opcional, documente la generalización a topologías arbitrarias y N interfaces (p. ej. *networkStar*). Explique cómo detectan interfaces activas y gates desconectadas. Incluya al menos un experimento adicional con figura o tabla. Si no la realizaron, elimine esta sección o indique brevemente por qué quedó fuera de alcance.

[Contenido opcional: topología estrella, desafíos de implementación, resultados comparativos con el anillo.]

A Guía para tablas y gráficos con datos de OMNeT++

Guía para completar: Este apéndice es solo orientativo: Les tiene que servir para saber cómo pasar de la simulación a evidencia publicable. Pueden eliminarlo en la entrega final.

A.1 Exportar resultados

1. Ejecute cada escenario con **semillas distintas** o repeticiones suficientes si busca intervalos de confianza.

Table 6: Comparación baseline vs. algoritmo propuesto (complete con sus datos).

Escenario / Métrica	Kickstarter		Propuesto	
	Media	Máx.	Media	Máx.
Caso 1 — Demora (s)	0.00	0.00	0.00	0.00
Caso 1 — Saltos	0.0	0	0.0	0
Caso 2 — Demora (s)	0.00	0.00	0.00	0.00
Caso 2 — Pérdidas	0	—	0	—

2. Exporte **escalares** (promedio, máximo, conteo) y **vectores** (series temporales) desde el IDE o con `scavetool`.
3. Guarde los CSV en `figuras/datos/` y documente en el paper la configuración (`omnetpp.ini`, tiempo de simulación, semilla).

Guía para completar: Ejemplo de flujo de trabajo: (1) simular → (2) exportar `.sca` y `.vec` → (3) procesar con script propio o planilla → (4) generar PDF/PNG → (5) incluir con `\includegraphics`. Mantenga los scripts en el repositorio junto al paper para reproducibilidad.

A.2 Ejemplo de tabla de mejoras porcentuales

Cuando compare algoritmos, una tabla de mejoras complementa la Tab. 6:

Table 7: Mejora relativa del algoritmo propuesto vs. kickstarter (ejemplo).

Métrica (escenario)	Base	Mej. (%)
Demora media (Caso 1)	0.82 s	−43%
Saltos promedio (Caso 1)	4.0	−50%
Demora media (Caso 2)	1.85 s	−50%
Ocup. máx. buffer (Caso 2)	25 pkts	−80%

Guía para completar: Calcule mejora como $(\text{baseline} - \text{propuesto}) / \text{baseline} \times 100$ para métricas donde **menor es mejor** (demora, saltos, pérdidas). Para throughput, invierta la fórmula. Explique en el texto si un −43% en demora es significativo en términos absolutos.

A.3 Cuándo usar cada recurso gráfico

A.4 Incluir figuras externas en L^AT_EX

Para gráficos generados con Python, R, Matplotlib o el IDE de OMNeT++:

```
\begin{figure}[!t]
\centering
```

Table 8: Recursos recomendados según el tipo de evidencia.

Objetivo	Recurso	Ejemplo
Comparar escenarios	Tabla + barras	Tab. 6
Sensibilidad param.	Curva x vs. métrica	Fig. 3
Evolución temporal	Serie de tiempo	Fig. 6
Distrib. demoras	Histograma	[si aplica]
Topología	Diagrama de red	Fig. 1

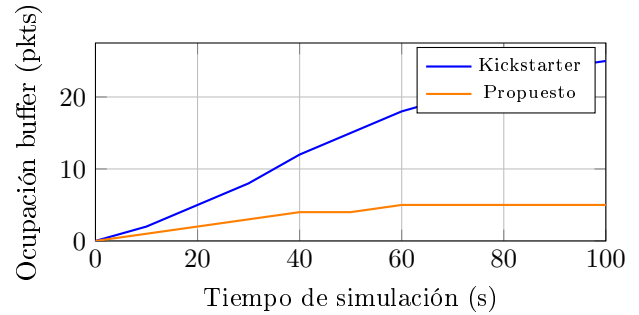


Figure 6: Ejemplo de serie temporal: ocupación de buffer. Útil para mostrar saturación o estabilidad en el Caso 2.

```
\includegraphics[width=\linewidth]{figuras/mi-grafico.pdf}
\caption{Demora media por escenario.}
\label{fig:mi-grafico}
\end{figure}
```

Guía para completar: Prefiera PDF o PNG de alta resolución. En el texto, **siempre** referencie la figura **antes** de que aparezca y explique qué se ve: no basta con insertar el gráfico sin interpretación.

B Checklist de entrega del paper

Guía para completar: Revise este listado antes de subir al repositorio. Borre el apéndice si ya no lo necesita.

- Título, autores, *abstract* e *index terms* en inglés.
- Cuerpo del artículo en español, sin cajas `\guia{}`.

- Respuestas explícitas a Caso 1, Caso 2 y evaluación comparativa.
- Al menos **3 figuras** y **2 tablas** con datos propios (no solo los ejemplos de la plantilla).
- Cada conclusión respaldada por una tabla o figura citada.
- Descripción del algoritmo, decisiones de diseño y detección de bucles.
- Extensión final entre 6 y 8 páginas en formato de dos columnas.
- Figuras con ejes, unidades, leyenda y caption descriptivo.
- Bibliografía en formato consistente (IEEE-like o APA).

B.1 Mapa entre tareas del laboratorio y secciones del paper

Table 9: Correspondencia entre el enunciado y las secciones de este documento.

Tarea / pregunta	Sección del paper
Instrumentar métricas	Sec. 2, Tab. 2
Caso 1: métricas y recursos	Sec. 3, Tab. 3
Caso 2: estabilidad	Sec. 3, Fig. 3
Diseño del algoritmo	Sec. 4
Comparación y bucles	Sec. 5
Tarea estrella (opcional)	Sec. 6

C Conclusiones

Guía para completar: Sintetice en un párrafo los hallazgos del análisis (Sección 3), las ventajas de su algoritmo (Sección 5) y aprendizajes del laboratorio. Evite introducir datos nuevos; apóyese en lo ya mostrado.

[Conclusiones concisas y respaldadas por las figuras/tablas del paper.]

[Párrafo sugerido: “En el análisis del kickstarter (Sección 3) observamos [...] (Tab. 3, Fig. 3). Nuestro algoritmo (Sección 4) redujo la demora media en un X% y evitó bucles mediante [...]. Como trabajo futuro proponemos [...].”]

Agradecimientos

Opcional: reconocimientos a la cátedra, compañeros o herramientas.

References

- [1] A. Varga and R. Hornig, “OMNeT++,” in *Proc. Int. Conf. Simulation Tools and Techniques (SimuTools)*, 2008.
- [2] Cátedra de Redes y Sistemas Distribuidos, “Laboratorio 4: kickstarter OMNeT++,” FaMAF/UNC, 2026.
- [3] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2011.